

Causal Inference and Machine Learning – Deep Learning, NLP, and Beyond

You’ve come a long way! Congratulations!

Chapter 11 marks an important turning point in our journey into causality. With this chapter, we’ll conclude our adventure in the land of causal inference and prepare to venture into the uncharted territory of causal discovery.

Before we move on, let’s take a closer look at what deep learning has to offer in the realm of causal inference.

We’ll start by taking a step back and recalling the mechanics behind two models that we introduced in *Chapter 9* – S-Learner and T-Learner.

We’ll explore how flexible deep learning architectures can help us combine the advantages of both models, and we’ll implement some of these architectures using the PyTorch-based CATENets library.

Next, we’ll explore how causality and **natural language processing (NLP)** intersect, and we’ll learn how to enhance modern Transformer architectures with causal capabilities, using Huggingface Transformers and PyTorch.

After that, we’ll take a sneak peek into the world of econometrics and quasi-experimental time series data, learning how to implement a Bayesian synthetic control estimator using CausalPy.

Ready?

In this chapter, we will cover the following:

- Conditional average treatment effects (ATEs) with deep learning
- Causal NLP with Transformers
- Bayesian synthetic control

Going deeper – deep learning for heterogeneous treatment effects

Since modern deep learning started gaining traction in the early 2010s, we have seen a continuous progression of breakthroughs. From AlexNet (Krizhevsky et al., 2012), which revolutionized computer vision, through Word2vec (Mikolov et al., 2013), which changed the face of NLP forever, to Transformers (Vaswani et al., 2017) and modern generative architectures (e.g. Radford et al., 2021, and Rombach et al., 2022), which fueled the generative AI explosion of 2022-2023.

Although the core idea behind (supervised) deep learning is associative in its nature and, as such, belongs to rung one of the **Ladder of Causation**, the flexibility of the framework can be leveraged to improve and extend existing causal inference methods.

In this section, we will introduce deep learning architectures to model **heterogeneous treatment effects** (aka **conditional treatment effects (CATE)**). We'll discuss the advantages of using deep learning to model CATE and implement two key architectures, using the PyTorch-based CATENets library (Curth & van der Schaar, 2021a).

CATE goes deeper

Differentiable deep learning architectures make it easy to arrange information flows in virtually any possible way. This opens the door to a whole new world of possibilities.

In *Chapter 9*, we introduced **S-Learner** and **T-Learner** algorithms. We saw that one of the advantages of S-Learner compared to T-Learner is sample efficiency.

S-Learner uses a single base-learner model and trains it with all available data. Meanwhile, T-Learner uses treated observations and control observations to train two separate base models.

One of the advantages of T-Learner is that by learning two separate response functions, it can be more effective in modeling these functions when they significantly differ.

What if we could combine the advantages of both models in one single architecture?

TARNet

The **Treatment-Agnostic Representation Network (TARNet)** is a neural network architecture, introduced by Uri Shalit from Technion (back then at NYU) and his colleagues in the 2017 ICML paper *Estimating individual treatment effect: generalization bounds and algorithms* (Shalit et al., 2017).

The architecture consists of a block of shared layers, trained using all available data, and two disjoint regression heads, trained using treated or control data, respectively. Intuitively, this architecture combines the sample efficiency of S-Learner thanks to shared layers and the flexibility of T-Learner thanks to disjoint regression heads.

Note that this solution also combats the greatest weakness of S-Learner – namely, the risk that a treatment variable will not be taken into account in the shared representation.

Figure 11.1 presents the TARNet architecture symbolically:

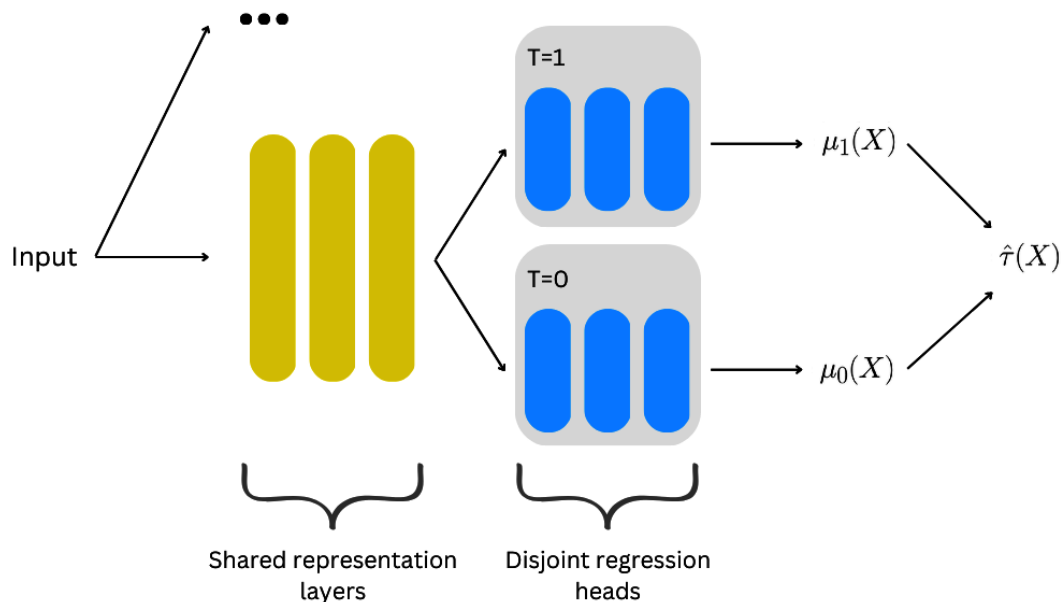


Figure 11.1 – A simplified TARNet architecture

As you can see in Figure 11.1, the first layers of TARNet (*shared representation layers*) leverage all the data. Then, the information flow is split between treatment- and control-specific heads (*disjoint regression heads*). The output from these heads are used to compute CATE.

In addition to what we see in Figure 11.1, TARNet uses an extra component that computes the discrepancy between the treatment and control covariate representations, which can be used to balance these representations when needed, analogously to the **propensity score** (marked symbolically by three dots in Figure 11.1, with details skipped for the clarity of presentation).

TARNet has been demonstrated to perform competitively or outperform a number of other architectures (Shalit et al., 2017, and Curth & van der Schaar, 2021a).

SNet

SNet (Curth & van Der Schaar, 2021a) is a deep learning-based architecture that can be thought of as a generalization of TARNet (and other architectures such as DragonNet (Shi et al., 2019) and DR-CFR (Hassanpour & Greiner, 2020)).

SNet consists of a set of parallel early layers trained on all data. In particular, SNet learns five different representations and feeds them to three disjoint heads – two regression heads (representing the outcome model) and one propensity score head (representing the treatment model).

The S in SNet's name comes from the fact that the information from the early representations in the network is shared between the latter task-specific heads.

Figure 11.2 presents the SNet architecture symbolically:

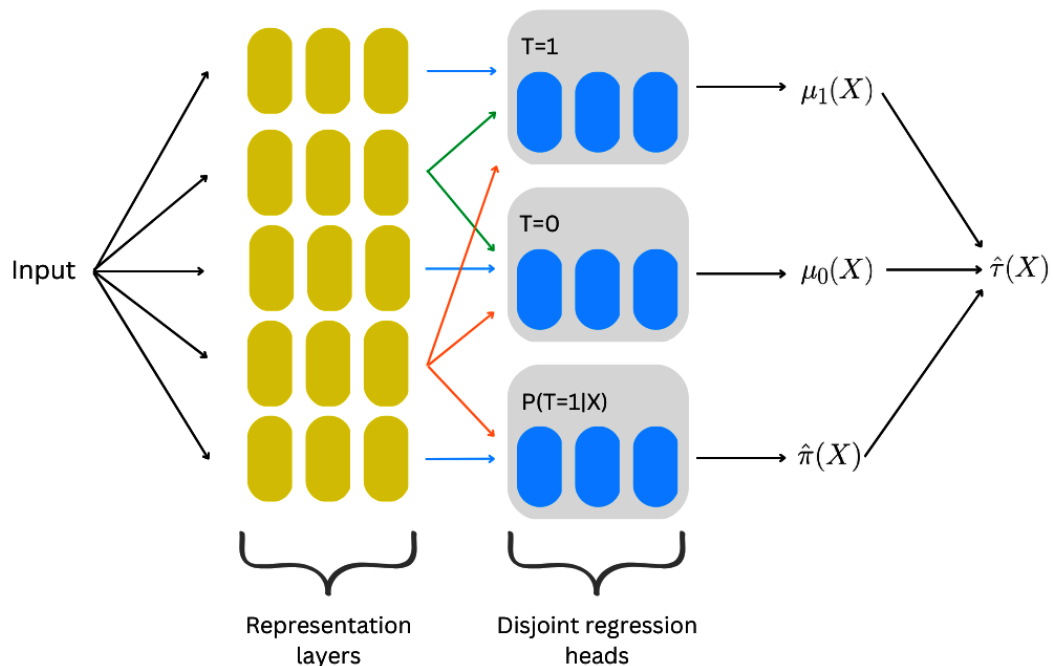


Figure 11.2 – A simplified SNet architecture

In Figure 11.2, we can see that the signal from each of the five representation layers (yellow) flows to a different set of regression heads. Blue arrows indicate flow to just a single head, while green and orange arrows indicate flow to multiple heads.

As all representations are learned jointly, the distinction between them might not be well identified. The authors add a regularization term to the model objective that “*enforces orthogonalization of inputs to (...) different layers*” (Curth & van der Schaar, 2021).

The authors demonstrate that SNet outperforms TARNet and neural network versions of T-Learner and DR-Learner over multiple synthetic and semi-synthetic datasets.

SNet’s additional flexibility might come at a price of more challenges when fitting the algorithm, as might be the case with any complex architecture.

We will now implement TARNet and SNet using the **CATENets** library.

Let’s meet CATENets.

CATENets

CATENets was developed by Alicia Curth, a researcher at the van der Schaar Lab at Cambridge University, led by Mihaela van der Schaar. The library is built on top of JAX (Bradbury et al., 2018) and provides a range of deep learning architectures to estimate CATE, including TARNet and SNet.

Additionally, the library includes implementations of three original algorithms proposed by Curth and van der Schaar – SNet (Curth & van der Schaar, 2021a), FlexTENet, and OffsetNet (Curth & van der Schaar, 2021b).

Most of the models are also available as PyTorch (Paszke et al., 2017) implementations (kindly contributed by Bogdan Cebere – <https://github.com/bcebere>). We’ll use these implementations in the subsequent code examples.

CATENets offers a very convenient and intuitive scikit-learn-style API.

PyTorch

PyTorch (Paszke et al., 2017) is an open source deep learning framework, originally developed by the Meta AI (Facebook AI) team and currently governed by the PyTorch Foundation. Version 2.0 was released in March 2023, introducing a number of features that can speed up PyTorch in numerous scenarios. In recent years, PyTorch has gained significant traction, especially in the research community. The models that we will experiment with in this chapter (CATENets and CausalBert) use PyTorch behind the scenes. In *Part 3, Causal Discovery*, we’ll build a complete PyTorch training loop when implementing Microsoft’s DECI model.

Experiments with CATENets

We’ll use a simulated non-linear dataset with a binary treatment for our CATENets experiments.

You can find the code for this and the next section in the `Chapter_11.1.ipynb` notebook (https://bit.ly/causal-ntbk-11_1).

Let's start with the data:

```
import numpy as np

SAMPLE_SIZE = 5000
TRAIN_SIZE = 4500
N_FEATURES = 20

X = np.random.normal(0, 1, (SAMPLE_SIZE, N_FEATURES))
T = np.random.binomial(1, 0.5, SAMPLE_SIZE)

weights = np.random.gumbel(5, 10, (SAMPLE_SIZE,
    N_FEATURES - 1))

y = (50 * T * np.abs(X[:, 0])**1.2) + (weights * X[:,
    1:]).sum(axis=1)

y0 = (50 * 0 * np.abs(X[:, 0])**1.2) + (weights * X[:,
    1:]).sum(axis=1)
y1 = (50 * 1 * np.abs(X[:, 0])**1.2) + (weights * X[:,
    1:]).sum(axis=1)
effect_true = y1[TRAIN_SIZE:] - y0[TRAIN_SIZE:]
```

We generate 5,000 samples and split them into training and test sets containing 4500 and 500 observations, respectively. Our dataset has 20 features. The data is generated according to the following formula:

$$Y_i = \left(50 * T_i * \text{abs}(X_i^{(0)})\right)^{1.2} + \sum_{d=1}^D w^{(d)} X_i^{(d)}$$

In the preceding formula:

- Y_i is the outcome of the i -th observation
- T_i is the treatment for the i -th observation
- $X_i^{(d)}$ is the d -th feature for the i -th observation
- $w^{(d)}$ is the weight for feature d
- $\text{abs}(\cdot)$ is the absolute value operator

We pick weights at random, drawing from the Gumbel distribution.

As you can see in the preceding formula, the treatment effect is non-additive and non-linear (we have an absolute value and raise the interaction term to the 1.2-th power). At the same time, only 1 out of 20 features we generated interacts with the treatment (feature $X^{(0)}$, which corresponds to $X[:, 0]$ in the code snippet).

We'll use our dataset to fit S-Learner, X-Learner, DR-Learner, and Causal Forest as a baseline. Next, we'll fit TARNet and SNet.

Let's start with the necessary imports:

```
from catenets.models.torch import TARNet, SNet
from econml.metalearners import S Learner,
from econml.dr import LinearDR Learner
from econml.dml import CausalForestDML
from lightgbm import LGBMRegressor
```

We imported the TARNet and SNet classes from `catenets.models.torch` and added Causal Forest DML and the S-, X-, and DR-Learners from EconML.

We'll also import PyTorch (`torch`) and PyTorch Lightning (`pytorch_lightning`):

```
import torch
import pytorch_lightning as pl
```

Let's set the computational device using `torch`:

```
device = 'cuda' if torch.cuda.is_available() else 'cpu'
device
```

If you have a GPU installed in your system that is supported by PyTorch and you have properly configured the drivers, this will print the following:

```
'cuda'
```

Otherwise, you should see the following:

```
'cpu'
```

Computations on CPU-only machines will take more time, but we prepared the datasets in a way that should allow you to complete them in a reasonable time on a CPU.

Next, let's set the seed for reproducibility. PyTorch Lightning offers a convenience function to do so:

```
SEED = 18
pl.seed_everything(SEED)
```

Note that even using `pl.seed_everything()` does not always lead to the same results. To enforce fully the deterministic behavior of PyTorch, we would need to use `torch.use_deterministic_algorithms(True)` and freeze the randomness in data loaders. At the time of writing, this would require modifying environmental variables and CATENets code. We won't go that far and keep the results *weakly reproducible*. Be prepared for your results to differ.

To learn more about reproducibility in PyTorch, check out <https://bit.ly/PyTorchReproducibility>.

Let's fit the benchmark models first:

```
benchmark_models = {
    'Slearner': Slearner(overall_model=LGBMRegressor()),
    'Xlearner': Xlearner(models=LGBMRegressor()),
    'DRlearner': LinearDRlearner(),
    'CausalForest': CausalForestDML()
}

benchmark_results = {}

for model_name, model in benchmark_models.items():
    model.fit(
        X=X[:TRAIN_SIZE, :],
        T=T[:TRAIN_SIZE],
        Y=y[:TRAIN_SIZE]
    )

    effect_pred = model.effect(
        X[TRAIN_SIZE:]
    )

    benchmark_results[model_name] = effect_pred
```

We create a dictionary of models and iterate over it, fitting each model and storing the results in another dictionary called `benchmark_results`. We used LightGBM regressors for S- and X-Learner and default models for DR-Learner and Causal Forest.

Now, let's fit TARNet and SNet:

```
tarnet = TARNet(
    n_unit_in=X.shape[1],
    binary_y=False,
    n_units_out_prop=32,
    n_units_r=8,
    nonlin='selu',
)

tarnet.fit(
    X=X[:TRAIN_SIZE, :],
    y=y[:TRAIN_SIZE],
    w=T[:TRAIN_SIZE]
)
```


We need to specify a number of parameters to initialize the TARNet class:

- `n_unit_in` defines the number of features in our dataset. To keep the code dynamic, we pass the size of the first dimension of our dataset as an argument. This parameter is mandatory.
- `binary_y` informs the model whether the outcome is binary. Our dataset has a continuous outcome, so we set it to `False`. Internally, this parameter will control which loss function the model should use for the outcome model (binary cross-entropy for a binary outcome and mean squared error otherwise). This parameter is set to `False` by default, but I wanted to use it explicitly because it's an important one.
- `n_units_out_prop` and `n_units_r` define the number of neurons in the propensity score layer (three dots in *Figure 11.1*) and representation layers (yellow layers in *Figure 11.1*), respectively. These parameters are optional, but I wanted us to see them, as they might be useful if you decide to tune TARNet for yourself one day. For more parameters (e.g., the number of layers), check out <https://bit.ly/TARNetCode>.
- `nonlin` defines the activation function used by the network. The default value is **exponential linear unit (ELU)**, but **scaled exponential linear unit (SELU)** has been shown to have an auto-normalizing effect on dense networks (Klambauer et al., 2017). This effect might not be guaranteed in more complex architectures such as TARNet, but I found SELU to work better than ELU for datasets similar to the one we use.

After initializing the model, we call the `.fit()` method and pass the data. Note that the naming convention in CATENets differs from the one we used in EconML:

- `X` takes the feature matrix (it should include all confounders if there is observational data)
- `y` takes the outcome vector
- `w` takes the treatment vector

We index the `X`, `y`, and `T` variables using the `TRAIN_SIZE` constant that we defined at the beginning (for example, `X[:TRAIN_SIZE, :]` means that we take all observations from the first one up to (but not including) the `TRAIN_SIZE` one and we take all the features (marked by the second colon after the comma)).

Note that we did not shuffle the dataset. This is because it's randomly generated.

Let's get predictions from TARNet:

```
effect_pred_tarnet = tarnet.predict(
    X=X[:TRAIN_SIZE, :]
).cpu().detach().numpy()
```

To get the predicted CATE, we use the `.predict()` method. Note that this time we pass the test dataset (starting from the `TRAIN_SIZE` observation up to the last one).

Note that we call a chain of methods on the result:

1. We call `.cpu()` to send the resulting tensor from a GPU (if you used one) to a CPU.
2. Next, we call `.detach()`, which detaches the tensor from the computational graph used by PyTorch behind the scenes (note that this graph does not have anything to do with causal graphs; it's used by PyTorch to perform computations efficiently).
3. Finally, we call `.numpy()` to cast the tensor to NumPy's `np.array` data type.

This chain is only necessary when we use a GPU, but it's harmless when working on a CPU-only machine.

Now, let's fit the SNet model.

The procedure is virtually identical to the one we used for TARNet:

```
snet = SNet(
    n_unit_in=X.shape[1],
    binary_y=False,
    n_units_out_prop=32,
    n_units_r=8,
    nonlin='selu',
)

snet.fit(
    X=X[:TRAIN_SIZE, :],
    y=y[:TRAIN_SIZE],
    w=T[:TRAIN_SIZE]
)
```

To get the predictions, we call the `.predict()` method and the CPU-detach-NumPy chain on the result:

```
effect_pred_snet = snet.predict(
    X=X[TRAIN_SIZE:, :]
).cpu().detach().numpy()
```

Great! Now we're ready to compare the results between all the models.

Figure 11.3 summarizes the results for TARNet, SNet, and our benchmark methods:

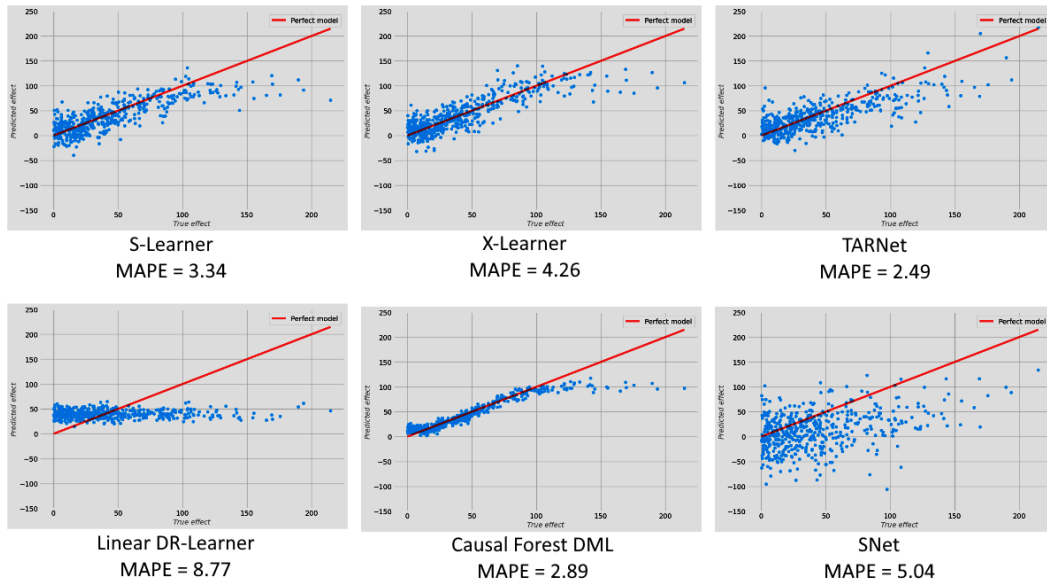


Figure 11.3 – Results for the benchmark models and deep learning models (TARNet and SNet)

As we can see in *Figure 11.3*, TARNet got the best results, achieving a **mean absolute percentage error (MAPE)** value of 2.49. SNet performed less favorably, with the highest variance out of all compared models. Perhaps the model could benefit from longer training and/or a larger sample size.

TARNet is the best at approximating low-density regions (the part in the top-right corner of each panel with a small number of points). Causal Forest DML has the lowest variance, but performs poorly in these low-density regions, giving place to TARNet.

S- and X-Learners seem to have variance comparable to TARNet, but they visibly underperform in comparison in the lower-density regions.

Finally, the linear DR-Learner fails to capture the heterogeneity of the effect. This is not surprising as the model – by definition – can only capture treatment effects linear in parameters.

Note that the results of our experiment should not be treated as a universal benchmark. We only ran one iteration for each model, but there's also a deeper reason.

As demonstrated by Curth et al. (2021), CATE models might perform very differently depending on the construction of the dataset. In particular, changing the function that modifies the treatment effect can favor some models over others.

This fact might make CATE model benchmarking on synthetic data challenging.

The authors propose a number of steps to address these challenges, such as using artificially biased experimental data to construct observational datasets. That said, each of their proposals comes with its own disadvantages (Curth et al., 2021).

In the real world, integrating expert knowledge into the process and validating model predictions with test data (using interventions and counterfactual predictions if possible) can be very helpful in addressing these challenges. This brings us to a seemingly paradoxical conclusion – it might be *easier* to evaluate a CATE model in a particular practical real-world use case than to compare its universal performance against other models on a curated benchmark.

Taking a broader perspective, perhaps we could say that this state of affairs is not unique to modeling causality, and contemporary non-causal machine learning faces similar challenges, yet they are harder to explicitly spot (for example, think about validating large language models trained on virtually all of the internet).

In this section, we introduced two deep learning-based architectures, TARNet and Snet, and implemented them using the CATENets library. We compared the results against other CATE methods and discussed the challenges in benchmarking CATE models.

In the next section, we'll see an example of adapting the Transformer architecture to causal NLP tasks.

Transformers and causal inference

“Der Gegenstand ist einfach.”

Ludwig Wittgenstein (1921)

It's 1916. The flames of war consume Europe. A young Austrian man of Jewish descent arrives at a hospital in Kraków, injured in an industrial explosion. He's a volunteer soldier who served in an Austrian artillery regiment.

There's something that differentiates him from other young men in the hospital.

His backpack is full of notes.

He keeps them close, but the notes are not a diary. They consist of a set of remarks on logic, ethics, language, and religion. Some of them were taken while he was still in the trenches of the Eastern Front.

The young man's name is Ludwig Wittgenstein, and his notes will later become the basis of the only book he will publish in his lifetime – *Tractatus Logico-Philosophicus* (Wittgenstein, 1922).

The book will become one of the most significant works of 20th-century Western philosophy.

One of the core ideas of the book is that most philosophical problems are created by the misuse of language. It states that fixing language by making clear references to real-world objects and states of affairs would solve existing philosophical problems – many by showing that they are mere artifacts of language misuse.

In this section, we'll discuss the challenges of using natural language in causal inference tasks, demonstrate one of the approaches to tackling this challenge, and implement it using a PyTorch-based CausalBert model.

The theory of meaning in five paragraphs

We started this section with a quote from Wittgenstein's book. The quote (thesis 2.02 of *Tractatus Logico-Philosophicus*) was translated into English by Charles Kay Ogden as "*The object is simple*" (Wittgenstein, 1922).

The theory of meaning proposed in the book states that names that we use in language refer to *simple* (atomic) objects. Atomic objects have no complexity; they cannot be divided further or described, only named.

There's beauty in this vision. The correspondence between language and the world seems simple and elegant. One challenge to this view is that in natural languages, the same word can often denote different objects, events, or even represent entire sentences.

For instance, the Arabic word **يالا** (*yalla*), used extensively by Arabic and Hebrew speakers, has numerous meanings that heavily depend on the context. It can mean (among others) the following:

- Let's go!
- Hurry up!
- Please go to bed now.
- Deal!
- Go away!
- Come here!
- Let's do it!

This context dependence is common in natural languages. Moreover, the changes in the meaning of a word might have a subtle character between different usages. Wittgenstein realized this and changed his approach toward meaning in the later stages of his philosophical career.

The new approach he took in his later works might have helped us build systems such as ChatGPT, but let's take it step by step.

Making computers understand language

Working with natural language based on discrete units that we call words has been a major challenge for computer scientists for a pretty long time.

Over the years, computational linguists and computer scientists came up with many creative ways to squeeze information from natural language documents. First attempts relied on bag-of-words-type approaches and information-theoretic analyses (e.g., Zipf in 1949).

The main challenge with these approaches was that they were not able to capture the aspects of semantic similarities between words (although some of them could capture some notion of similarity on a document level).

Wittgenstein's approach toward meaning moved from the idea of words denoting objects to contextual usage. In *Philosophical Investigations* (published posthumously), Wittgenstein (1953) wrote, “*The meaning of a word is its use in the language*,” and proposed that different meanings might be codified within **language games**.

These ideas are closely related to the notion promoted by English linguist John Rupert Firth, summarized as, “*You shall know a word by the company it keeps*” (Firth, 1957).

Wittgenstein's works have likely influenced many lines of research in artificial intelligence and linguistics through a number of paths. One of these paths led to the works of his student Margaret Masterman, who applied Wittgensteinian ideas to the realm of machine translation. She founded the Cambridge Language Research Unit in 1955, and her work influenced researchers around the world (Molino & Tagliabue, 2023).

From philosophy to Python code

A 2013 seminal paper by Czech Google researcher Tomáš Mikolov and colleagues changed the face of NLP forever. The paper presented a self-supervised algorithm called **word2vec** that learned continuous dense vector representations of words, allowing the framing of semantic similarity as a distance between vectors in a continuous multidimensional space (Mikolov et al., 2013).

Although *word2vec* was not able to model **polysemy** (multiple meanings of the same word), it can be considered the first practical implementation of the core ideas proposed in Wittgenstein's later works.

Less than five years after the publication of the *word2vec* paper, **ELMo** – a model introduced by Matthew E. Peters and colleagues (Peters et al., 2018) – made modeling polysemy possible. Just a couple of months later, another model – **BERT** – was released by a team at Google (Devlin et al., 2018). BERT replaced recurrent neural networks with a multi-head attention mechanism (Vaswani et al., 2018). BERT is an example of the Transformer architecture that also gave birth to the GPT family of models (Radford et al., 2019, Brown et al., 2020, and OpenAI, 2023).

BERT has revolutionized the entire field of NLP, bringing huge improvements to most if not all known benchmarks. Its generative cousins from the GPT family made NLP a mainstream topic with media headlines.

Although modern **large language models (LLMs)** such as BERT or GPT-4 led to an impressive leap in what's possible in today's natural language modeling and processing, they have some limitations.

LLMs and causality

ChatGPT is a chatbot system introduced by OpenAI in November 2022 (OpenAI, 2022). The system is based on the GPT-3.5 and GPT-4 models, trained using the **reinforcement learning from human feedback (RLHF)** paradigm. ChatGPT is well known to produce plausible sounding short-, mid-, and even long-form texts.

It has been demonstrated that the model can successfully answer various types of causal and counterfactual questions. *Figure 11.4* presents how ChatGPT (correctly) answered the counterfactual query that we solved analytically in *Chapter 2*.

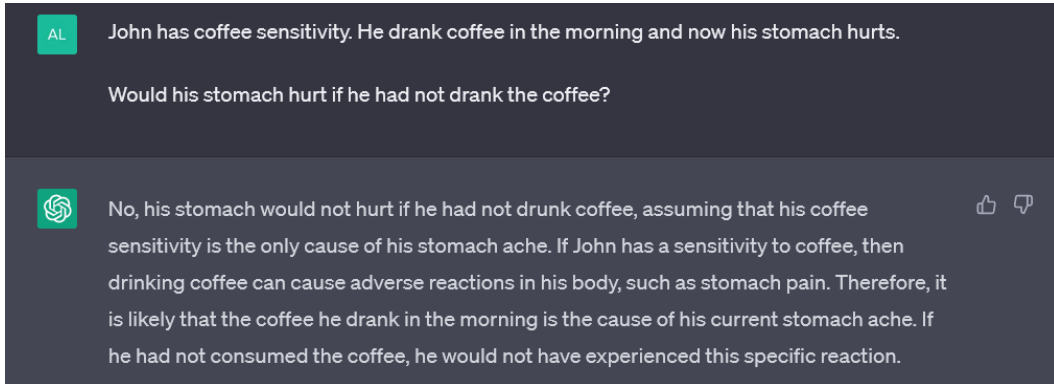


Figure 11.4 – ChatGPT’s counterfactual reasoning

I find the behavior presented in *Figure 11.4* impressive, yet it turns out that under more systematic examination, the model is not entirely consistent in this realm.

The system has been thoroughly scrutinized from the causal perspective by two independent teams at Microsoft (led by Cheng Zhang) and TU Darmstadt (led by Moritz Willig and Matej Zečević). Both teams arrived at similar conclusions – the system has significant potential to answer various types of questions (including some causal questions), but cannot be considered fully causal (Zheng et al., 2023 and Willig et al., 2023).

Willig et al. (2023) proposed that ChatGPT learns a meta-SCM purely from language data, which allows it to reason causally but only in a limited set of circumstances and without proper generalization.

Zhang et al. (2023) suggested that the model could benefit from integration with implicit or explicit causal modules. Her team has even demonstrated an early example of GPT-4 integration with a causal end-to-end framework DECI (we will discuss DECI in *Part 3, Causal Discovery*).

In a more recent paper titled *Causal Reasoning and Large Language Models: Opening a New Frontier for Causality* (Kıcıman et al., 2023), Emre Kıcıman and colleagues demonstrated that GPT-based models outperform many existing models on pair-wise causal discovery, counterfactual reasoning, and in determining necessary and sufficient causes, with GPT-4 achieving an accuracy of 92.44% on the CRASS counterfactual reasoning benchmark (comparing to the 98.18% accuracy of human annotators).

Figure 11.5 presents an accuracy comparison of the CRASS counterfactual benchmark (Frohberg & Binder, 2022) between different models and human annotators.

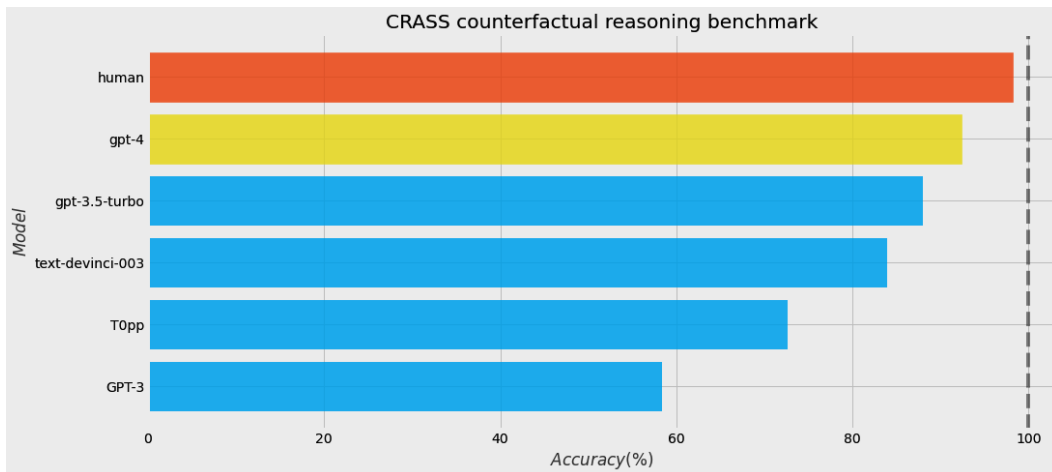


Figure 11.5 – Performance of models and humans on the CRASS counterfactual benchmark

These results are truly impressive. The main challenge that remains is related to model failure modes, which are difficult to predict and might occur “*even in tasks where LLMs obtain high accuracy*” (Kıcıman et al., 2023). This might be related to the fact that the *meta-SCM* that the model learns (as proposed by Willig et al. (2023)) is correlational in its nature, so the model might produce mistakes stochastically.

Keeping in mind that LLMs are not yet capable of systematic causal reasoning, a question that naturally arises is whether there are other ways in which they can be useful in the context of causality.

It turns out that the answer is positive.

The three scenarios

There are two main types of questions we can ask at the intersection of NLP and causality:

- Questions regarding LLMs’ and/or other NLP tools’ capabilities to help us answer causal queries about the world (e.g., did a writing training program improve the participants’ writing clarity?)
- Questions regarding the internal causal mechanics of LLMs or other models (e.g., what changes in the output can be caused by altering the embedding space?)

We’ll focus solely on the first type of question.

There are three basic scenarios where we can leverage LLMs and other NLP tools to answer causal queries when addressing the first type of question. In each of the scenarios, we will use text as a node (or a set of nodes) in a causal graph.

The three scenarios are as follows:

- Text as a treatment
- Text as an outcome
- Text as a confounder

In this subsection, we will draw from a great review article by Amit Feder and colleagues, which I wholeheartedly recommend you read if you're interested in causality and NLP (Feder et al., 2020).

Let's review the three scenarios.

Text as a treatment

Hanna is a copywriter at a top New York marketing agency. Over the last few weeks, she's been working on a high-profile project. She and her team are working on an online campaign to drive conversions for a major fashion brand.

Hanna has poured her heart into the project, painstakingly choosing words that she believes will speak to her ideal client. She's created numerous successful campaigns in the past, but deep down in her heart, she really doesn't know which aspects of her copywriting are responsible for their success.

Is it the wording? The use of metaphors? The rhythm of the copy? A combination of all of these things?

Hanna relies on her intuition to decide when the copy is good. She's often right, but quantifying the impact of the copy in a more systematic way could help her work and scale to new niches much faster. It would also give her an excellent asset that she could leverage in communication with internal and external stakeholders.

One of the examples of how Hanna's use case could be addressed comes from Reid Pryzant et al.'s paper (2017). The authors designed a neural network to isolate language aspects that influence sales and concluded that in their sample (coming from a Japanese online marketplace Rakuten), appealing to authority by using polite, informative, and seasonal language contributed most to increased sales.

Some other works that aim at discovering the features of language that impact the outcome of interest include the discovery of conversational tendencies that lead to positive mental health outcomes (Zhang et al., 2020).

These works are very interesting and open a path toward new fascinating research directions. At the same time, they come with a number of challenges.

For instance, it might be difficult to exclude confounding that is rooted internally in text. Text is a source of the treatment, but other (non-treatment) aspects of text might impact the treatment as well as the outcome (Feder et al., 2022).

Moreover, the situation can be further complicated by the fact that different readers might interpret a text differently (to see an example of a model that takes the reader's interpretation into account, check out Pryzant et al. (2021)).

Text as an outcome

Yizé is an aspiring writer. He has a prolific imagination and produces stories with ease, but he feels that the way he writes lacks a bit of focus and clarity. He decides to enroll in a writing course at a local community college.

Measuring Yizé's improvement in writing clarity after the course is an example of a text-as-outcome scenario.

In this scenario, the treatment can be relatively easily randomized (technically speaking), and deconfounding the observational data could be possibly much easier than in the previous case.

The main challenge in this scenario is the measure of the outcome. If we use a model to measure clarity and this model is trained on the data from our sample, the measure of clarity becomes dependent on the values of treatment for all participants. This makes the outcomes of all participants dependent on the treatments of all other participants, which violates the consistency assumption (Feder et al., 2022).

One way to deal with this challenge is to train the outcome measurement model on another sample (Egami et al., 2018). This can be relatively easily done if we have enough samples.

Text as a confounder

Finally, text can also be a confounder.

Catori and her friend Stephen both love manga and are inspired by similar characters. They pay attention to similar details and often find themselves wanting to share the same story at the same moment.

Both of them post about manga on Reddit. One day, they noticed an interesting pattern – Stephen's posts get more upvotes, and it's much more likely that his post gets an upvote within the first hour after being posted than Catori's post does. They are curious about what causes this difference.

It has been repeatedly shown that across scientific fields, female authors are cited less frequently than their male counterparts. This effect has been demonstrated in neuroscience (Dworkin et al., 2020), astronomy (Caplar et al., 2017), transplantation research (Benjamens et al., 2020), and so on.

Could the fact that Catori is perceived as female impact how other Reddit users react to her posts?

As seasoned causal learners, we know that to answer this question, we need to carefully consider a number of factors.

Catori's gender might impact the topics she chooses, her style of writing, or the frequency of using certain words. All of these choices can impact the way other participants react to her content.

Moreover, other Reddit users do not know Catori's true gender. They can only infer it from her profile information, such as her username or her avatar.

Therefore, the question here is about *perceived* genders, rather than the true gender's impact on other users' behavior.

Let's build a **directed acyclic graph (DAG)** representing this problem. We'll also add a node that represents whether a given post contains a picture or not.

Figure 11.6 presents the proposed DAG:

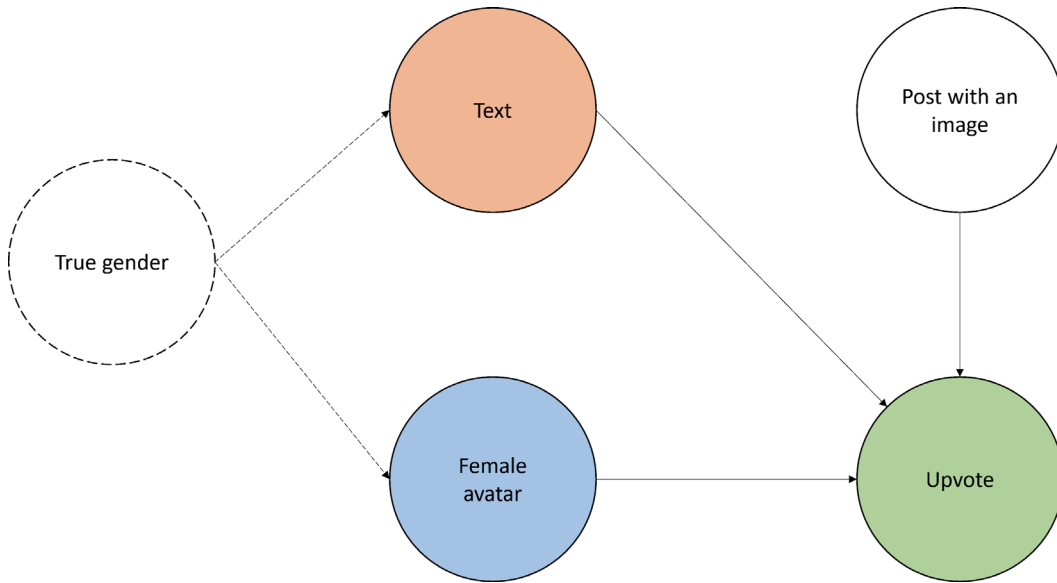


Figure 11.6 – The Reddit DAG

The blue node in Figure 11.6 represents our treatment (*perceived gender*), and the green one is the outcome (*Upvote* – a post upvote within one hour of posting).

True gender impacts the probability that a person will use a female avatar and impacts *Text* (e.g., the topic or linguistic properties). Note that *True gender* is unobserved, which we indicate by dashed lines in Figure 11.6.

Additionally, we have a node indicating whether a post contains an image. In our model, this node is independent of *true* and *perceived gender*.

Note that the *True gender* node opens a backdoor path between the treatment (*Female avatar*) and outcome (*Upvote*). Luckily, its impact is mediated by *Text*. This means that by controlling for *Text*, we can block the path.

A natural question to ask here is which aspects of the text we should control for and how can we make sure that they are present in the representation of the text that we choose to use.

CausalBert

CausalBert is a model proposed by Victor Veitch, Dhanya Sridhar, and David Blei from Columbia University in their paper *Adapting Text Embeddings for Causal Inference* (Veitch et al., 2020). It leverages BERT architecture (Devlin et al., 2018) and adapts it to learn **causally sufficient embeddings** that allow for causal identification when text is a confounder or mediator.

CausalBert is conceptually similar to DragonNet (Shi et al., 2019) – a descendant of the TARNet architecture. *Figure 11.7* presents CausalBert symbolically:

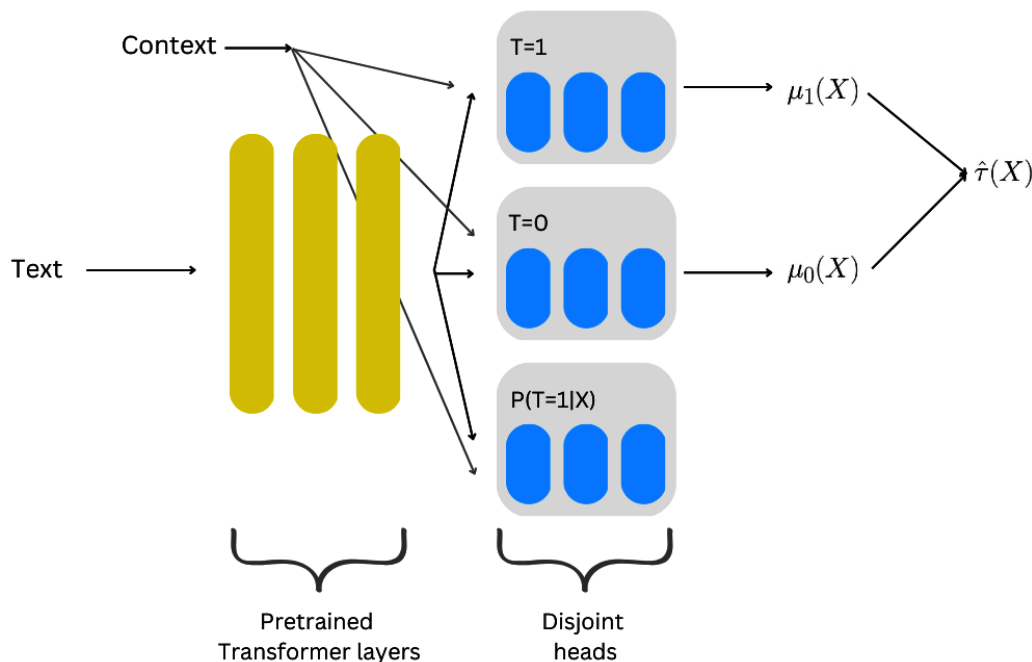


Figure 11.7 – CausalBert architecture

The yellow layers in *Figure 11.7* are pretrained Transformer layers. These layers come from the BERT model pre-trained on the original corpus. Three disjoint heads estimate the two potential outcomes and the propensity score, all starting from the same shared representation.

The BERT model family

Since the original BERT model was introduced in 2018, many modifications of the original architecture have been proposed. One of them is DistilBERT (Sanh et al., 2019) – a smaller, faster, and lighter version of BERT that retains about 97% of the original model's capabilities, while reducing its size by 40% and performing computations in 60% less time. CausalBert uses DistilBERT behind the scenes.

Note that the version of CausalBert presented in *Figure 11.7* is slightly different from the original version proposed by Veitch et al. (2020) and follows Reid Pryzant's PyTorch implementation. Pryzant added the possibility to include an additional categorical confounder (or context) variable, marked as *Context* in the figure.

During the training, CausalBert adapts the pretrained layers and learns the embedding, outcome, and treatment models jointly. This joint objective preserves the information in the embeddings that is predictive of the treatment and outcome and attenuates everything else. Thanks to this mechanism, we make sure that information relevant to the downstream task is preserved in the embeddings, making them sufficient to effectively control for confounding.

Note that the same mechanism is helpful when the text is a partial mediator of the treatment effect. When controlling for text in this scenario, we learn the direct effect of the treatment on the outcome (this will exclude all the information that travels through the mediator and only leave the information that flows directly from the treatment to the outcome, also known as the **natural direct effect (NDE)**).

Playing safe with CausalBert

The mechanism that adapts the embeddings that predict the treatment and outcome also has a darker side. Confounding and mediation scenarios are not the only ones in which text is correlated with both the treatment and outcome.

It will also happen when text is a common descendant of the treatment *and* outcome. In such a case, the text becomes a collider, and controlling for it opens a spurious path between the treatment and outcome, biasing the results.

To mitigate this risk, we should always make sure that none of the aspects of the text are a descendant of the treatment *and* outcome; neither should any aspect of the text be a descendant of the outcome alone (as this would nullify any effect of the treatment on the outcome).

For instance, if a Reddit user shares a post, observes a lack of upvote within the first 15 minutes after posting, and then edits the post in the hope of getting an upvote, the text becomes a collider between the treatment and the outcome.

On top of this, CausalBert requires standard causal inference assumptions – a lack of hidden confounding, positivity, and consistency (one type of treatment and no treatment interference between subjects; check out *Chapter 8* for more details).

CausalBert in code

We will implement CausalBert using Reid Pryzant's PyTorch implementation (<https://bit.ly/CausalBertCode>).

At the time of writing, the code in the original repository contained a small mistake that could distort the model's results. A fixed version of the code that we'll use in this section can be found in the book's repository (<https://bit.ly/CausalBertCodeBook>).

Let's start with the imports:

```
import pandas as pd
from models.causal_bert_pytorch.CausalBert import CausalBertWrapper
```

We import `pandas` to read the data and the `CausalBertWrapper` class, which implements the model and wraps it in a user-friendly API.

Next, we read in the data:

```
df = pd.read_csv('data/manga_processed.csv')
```

Our dataset has been generated according to the structure presented in *Figure 11.6*. It consists of 221 observations and 5 features:

- `text` – the text content
- `subreddit` – the subreddit name (we'll ignore this variable, as it has a single value for all posts)
- `female_avatar` – a binary variable, indicating whether a user has a female avatar in their profile
- `has_photo` – a binary indicator if a post contains an image
- `upvote` – a binary indicator of a post upvote in the 1st hour after posting (the outcome)

Texts are short Reddit-like posts, generated using ChatGPT and prompted indirectly to produce gender-stereotypical content. Both the *text* and *female avatar* indicators are confounded by an unobserved *true gender* variable, and the *text* causally impacts the outcome through linguistic features (for the data-generating process code, check out <https://bit.ly/ExtraDGPs>).

Figure 11.8 presents the first five rows of the dataset before shuffling:

	text	subreddit	female_avatar	has_photo	upvote
0	What's your favorite romance manga with a fema...	manga	1	0	1
1	As a woman, I find 'Nana' to be a very empower...	manga	1	0	0
2	Can we talk about how badass the female charac...	manga	1	0	0
3	Any other ladies here obsessed with 'Ouran Hig...	manga	1	1	0
4	Just finished reading 'Skip Beat!' and I'm in ...	manga	1	1	1

Figure 11.8 – A manga pseudo-Reddit dataset

Let's instantiate the model:

```
causal_bert = CausalBertWrapper(
    batch_size=8,
    g_weight=0.05,
```

```
Q_weight=1.,  
mlm_weight=0.05  
)
```

We pass four parameters to the constructor.

We define the batch size and three `weight` parameters. These three parameters will be responsible for weighting the components of the loss function during the training.

Here's their meaning:

- `g_weight` is responsible for weighting the part of the loss that comes from propensity score (the " $P(T=1|X)$ " block in *Figure 11.7*; it's called g , according to the convention used in Veitch et al.'s (2018) paper)
- `Q_weight` is responsible for weighting the outcome model loss (the sub-model that predicts the actual outcome – the " $T=0$ " and " $T=1$ " blocks in *Figure 11.7*, called Q according to the paper's convention)
- `mlm_weight` weights BERT's **masked language model (MLM)** loss

MLM

BERT and some of its cousins are trained using the so-called MLM objective. It's a self-supervised training paradigm in which a random token in a training sequence is masked, and the model tries to predict this masked token using other tokens in the sequence. Thanks to the **attention mechanism**, the model can "see" the whole sequence when predicting the masked token.

As you can see, we weighted the outcome model loss (`Q_weight`) much higher than the propensity model (`g_weight`) or MLM loss (`mlm_weight`).

This setting works pretty well for our dataset, but when you work with your own data, you should tune these parameters only if you have enough samples to do so.

The low weighting of the MLM loss indicates that the texts in our dataset are likely not very different from the overall language on the internet, and the generic DistilBERT embeddings provide us with a good-enough representation for our task.

Let's train the model:

```
causal_bert.train(  
    texts=df['text'],  
    confounds=df['has_photo'],  
    treatments=df['female_avatar'],  
    outcomes=df['likes'],  
    epochs=6  
) ;
```

The `.train()` method takes five arguments:

- `texts` – an array (`pd.Series`) of texts
- `confounds` – a categorical array containing values of an additional confounding or context variable (if there's no such variable in your setting, just pass an array of zeros here)
- `treatments` – a binary treatment indicator
- `outcomes` – a binary outcome indicator
- `epochs` – the number of training epochs

After training the model, we can compute the ATE.

To do so, we call the `.inference()` method and pass texts and additional confounds as arguments. This method returns a tuple. We're interested in the 0th element of this tuple:

```
preds = causal_bert.inference(
    texts=df['text'],
    confounds=df['has_photo'],
)[0]
```

The 0th element contains an array of dimension $N \times T$, where N is the number of observations and T is the number of treatment levels (for a binary treatment $T = 2$). Entries in the first (0th) column are the probabilities that the outcome is equal to 1 under no treatment. Entries in the second (1st) column are the probabilities that the outcome is equal to 1 under the treatment.

To calculate the ATE, we subtract the entries in the first column from the entries in the second column and take the average:

```
np.mean(preds[:, 1] - preds[:, 0])
```

This gives us the following:

```
-0.62321178301562
```

This result is pretty close to the true effect of -0.7.

It indicates that perceived gender causes a significant drop in early reactions to the content published in our imaginary manga subreddit.

This conclusion is only valid if the three causal assumptions (no hidden confounding, positivity, and consistency) are met and the model is structurally sound (no aspects of the text are descendants of the treatment and outcome, nor the outcome itself).

If Catori and Stephen had doubts regarding some of these assumptions, they could run a randomized experiment on Reddit to see whether the results held.

Before we conclude this section, I want to take a step back and take a look at the topic of **fairness**. At the beginning of this section, we cited a number of papers, demonstrating a citation gender gap between female and male authors.

Although we might be tempted to make conclusions regarding gender gaps and other similar effects based on simple statistical analyses of observational data (e.g., survey data), this is usually a bad idea.

The structural aspect is crucial in fairness analyses, and ignoring it will *almost surely* lead to invalid results (including sign reversal, effect masking, or hallucinating non-existent effects). To learn more about this topic, check out the excellent works of Drago Plečko and Elias Bareinboim (Plečko & Bareinboim, 2023).

In this section, we explored the intersection of NLP and causality, starting with a discussion on the ideas of Ludwig Wittgenstein and how these and similar ideas contributed to making computers encode natural language efficiently.

We then delved into the three types of scenarios encountered at the intersection of NLP and causality – text as a treatment, text as an outcome, and text as a confounder. Next, we demonstrated an approach to adapt text embeddings for causal inference in the text-as-confounder scenario, using a Transformer-based CausalBert model.

Finally, we discussed the limitations and potential challenges that we can encounter when working with this model. We finished the chapter by sharing references to materials that discuss how we can tackle the problems of fairness from a causal perspective.

In the next section, we'll take a look at one of the ways in which we can leverage the time dimension to draw causal conclusions when experiments are not available.

Causality and time series – when an econometrician goes Bayesian

In this section, we're going to introduce a new style of thinking about causality.

We'll start this section with a brief overview of quasi-experimental methods. Next, we'll take a closer look at one of these methods – the **synthetic control estimator**. We'll implement the synthetic control estimator using an open source package, **CausalPy**, from PyMC Labs and test it on real-life data.

Quasi-experiments

Randomized controlled trials (RCTs) are often considered the “*gold standard*” for causal inference. One of the challenges regarding RCTs is that we cannot carry them out in certain scenarios.

On the other hand, there's a broad class of circumstances where we can observe naturally occurring interventions that we cannot control or randomize. Something *naturally* changes in the world, and we are interested in understanding the impact of such an event on some outcome of interest.

Events like this are sometimes referred to as **natural experiments**.

Such interventions do not have the power to guarantee unconfoundedness, yet sometimes, they can provide us with useful information.

A family of methods traditionally used to analyze data coming from natural experiments (also known as quasi-experiments) is known as **quasi-experimental methods**.

Traditionally, quasi-experiments are framed as time series problems. An event is observed at some point in time, and we track how some measurable quantity changes after this event compare to some baseline.

Let's make it more concrete.

Twitter acquisition and our googling patterns

On October 27, 2022, Elon Musk acquired a popular social media platform, Twitter.

The acquisition process was lengthy and marked by numerous controversies.

The day after the formal acquisition, on October 28, Musk tweeted, “*The bird is freed*,” suggesting that the acquisition had now become a reality. The tweet quickly gained significant attention from the public and media outlets alike, with many people commenting and resharing it on the platform.

Significant events in politics, economy, or art and culture can spark public attention and alter the behaviors of large groups of people, making them search for additional information.

Was Twitter's acquisition such an event?

Let's see.

To answer this question, we'll take a look at Google Trends data and compare how often users searched for Twitter before and after Elon Musk's tweet.

To perform such a comparison, we will need some baseline.

The logic of synthetic controls

With RCTs, we randomly assign units to the treatment and control groups and compare the outcomes between both groups to quantify the relative efficacy of the treatment.

In quasi-experiments, we don't have control over the treatment assignment. Moreover, the intervention is oftentimes limited to just one unit.

How can we navigate this?

First, let's notice what we already have. We have a treated unit, and we believe that we know at which point in time the treatment has been assigned.

That's valuable information, but it's not sufficient to draw conclusions.

To make a comparison, we'll also need a baseline – a control group or control unit.

A useful control unit would provide us with information about what would have happened to the treated unit if the treatment had not occurred. As we already know, this alternative outcome cannot be observed. How can we deal with this?

Recall the core idea behind meta-learners. With meta-learners, we learn response functions for treated and untreated units and then predict one or both outcomes, based on the learned representation. We use these counterfactual outcomes to compute the estimated treatment effect.

Synthetic control is different from meta-learners in a number of significant aspects, yet it bears a conceptual similarity to them.

In synthetic control (Abadie & Gardeazabal, 2003), we observe units over time. If two units are correlated over an extensive period of time, it is highly likely that either one of the units has a (direct or indirect) causal effect on the other unit, or they have a common cause that is a source of variability in both of them.

Whichever scenario of the two is the case, the two units will have some predictive power to predict each other (as long as the underlying data-generating process does not change).

Reichenbach's principle

Reichenbach's common cause principle asserts that when two variables are correlated, there must be either a causal relationship between the two, or a third variable exists (known as a Reichenbachian common cause) that causes the correlation between the two.

In a finite data regime, correlations between two variables might also occur randomly. That's why in the text we say that it's *highly likely* that the two variables are causally related or they have a common cause, rather than saying that this is *certainly* true. Random correlations between two long time series are highly unlikely, but the shorter the series, the more likely the random correlation is.

Note that some authors propose (putative) counterexamples to Reichenbach's principle (check out <https://bit.ly/ReichenbachPrinciple> for more details). The principle is also debated in quantum physics (the Einstein-Podolsky-Rosen argument – e.g., Rédei, 2002).

The main idea behind synthetic control is to find units that are correlated with our unit of interest in the pre-treatment period, learning a model that effectively predicts the behavior of the treated unit after the treatment occurs. The units that we use as predictors are called the **donor pool**.

Let's unpack it.

A visual introduction to the logic of synthetic controls

Let's take a look at *Figure 11.9*, which presents an outcome variable (**Target**, in blue) and a set of potential donor pool variables (green, red, and purple). Note that all variables are recorded over time:

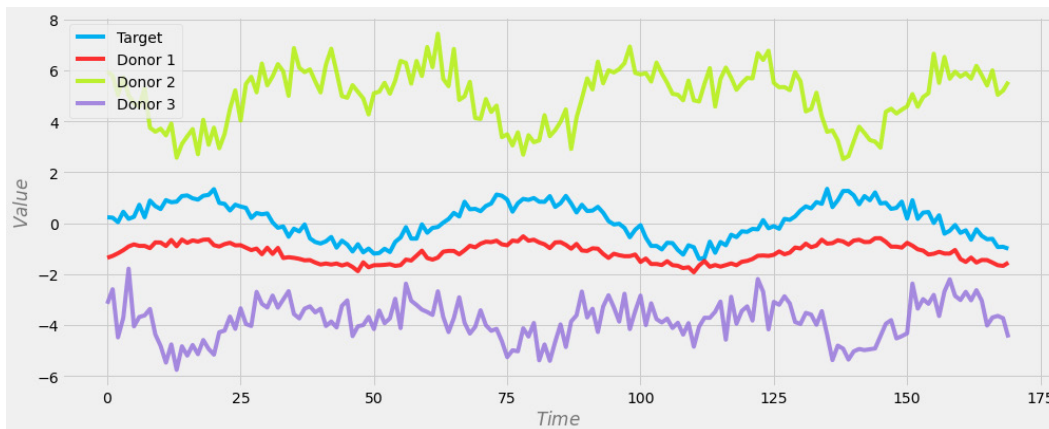


Figure 11.9 – An outcome variable (Target) and a set of potential donor variables

We're aiming to predict the outcome variable (**Target**) by using the remaining variables (**Donor 1**, ..., **Donor 3**) as predictors.

The donor pool variables in *Figure 11.9* seems to be good potential predictors of **Target**, as there are visible correlation patterns between them and **Target**.

To see this in a more systematic way, *Table 11.1* presents Pearson correlation coefficients between the donor pool variables and the outcome.

Variable	Pearson's r (donor-outcome)	p-value
Donor 1	0.92	<0.0001
Donor 2	-0.60	<0.0001
Donor 3	-0.33	<0.0001

Table 11.1 – The correlation coefficients between the donor variables and the outcome

Strong significant correlation coefficients indicate that these donor pool variables will be good predictors of the outcome.

Let's see how well we can predict the outcome from these variables using simple linear regression. *Figure 11.10* presents the actual outcome variable (blue) and its predicted version (gray):

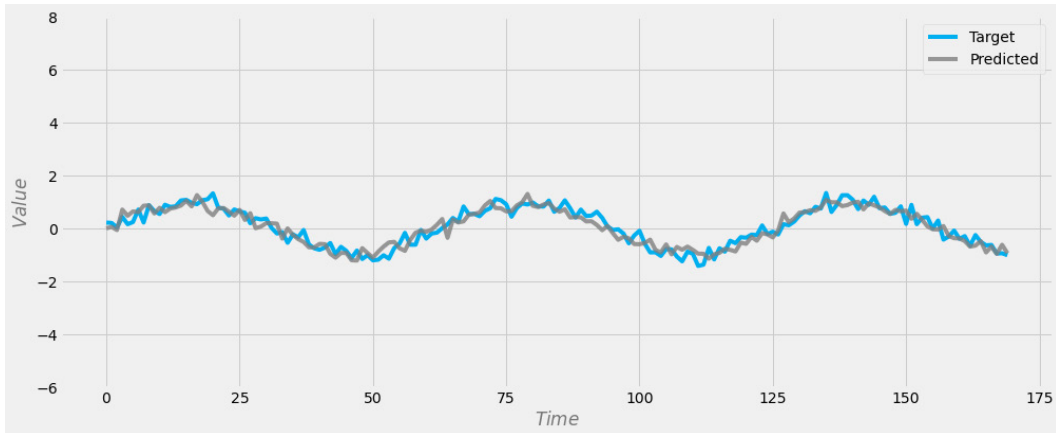


Figure 11.10 – The actual target and prediction based on donor variables

As we can see, the overall fit seems really good.

An important property of donor pool units is that they should be correlated with the treated unit in the pre-treatment period (before the treatment occurs), but they should not react to the treatment themselves.

In *Figure 11.11*, we can see a longer version of the time series from *Figure 11.9* with a treatment recorded at time **170**:

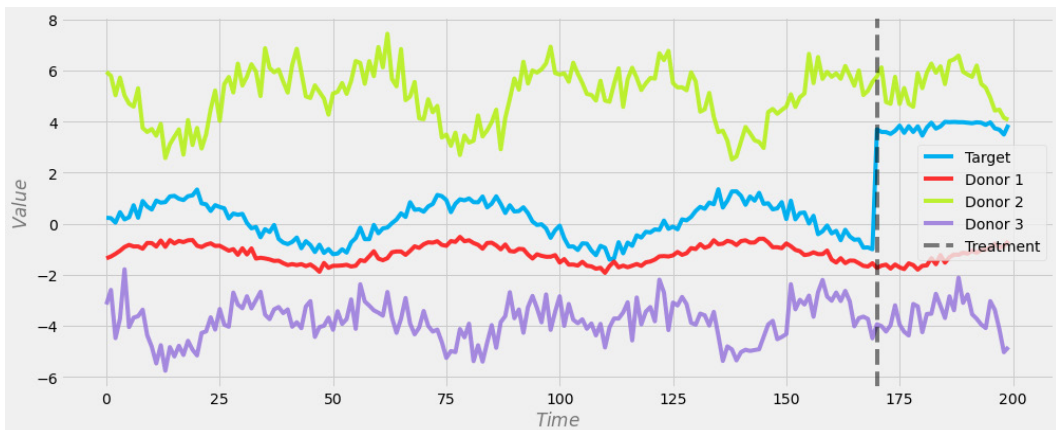


Figure 11.11 – An outcome variable (Target), a set of donor pool variables, and the treatment

As we can see, our outcome variable changed significantly after the treatment occurred, but donor variables seem to follow their previous pattern in the post-treatment period.

Let's predict the outcome variable from the donor pool variables after the occurrence of the treatment.

Figure 11.12 presents the results:

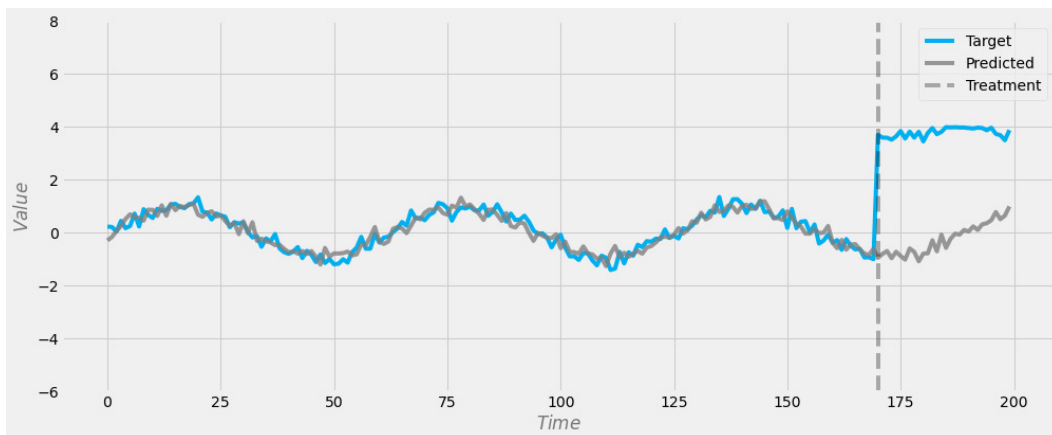


Figure 11.12 – The actual target and prediction, based on donor pool variables in the pre- and post-treatment periods

The prediction (the gray solid line in Figure 11.12) in the post-treatment period is our *synthetic control* unit.

Having the *actual outcome* (the blue line in Figure 11.12) and the predicted *counterfactual outcome* (the gray line in Figure 11.12), we can subtract the latter from the former to get the estimated treatment effect at any point in the post-treatment period.

Conceptually, we learn the counterfactual outcome from the donor pool variables, and then we subtract the values of this counterfactual outcome from the values of the actual outcome to obtain the estimate of the treatment effect.

Let's use this logic to see whether Elon Musk's tweet had enough impact to alter our Google search patterns.

Starting with the data

In this section, we'll use Google Trends data on search volume for different social media platforms over time.

We'll use the information on search volume for LinkedIn, TikTok, and Instagram as our donor pool units. We sampled the data with daily granularity between May 15, 2022 and November 11, 2022, with the treatment (the tweet) occurring on October 28, 2022. This gives us 181 samples (days) in total, with 166 samples (days) in the pre-treatment period. This sample size should be more than sufficient to capture predictive regularities in the dataset.

Synthetic controls in code

In the visual introduction section, all that we did was based on a simple linear regression model. In fact, the synthetic control estimator is based on the same idea.

The synthetic control estimator learns to predict the outcome variable using the donor pool variables in the pre-treatment period, and then it predicts the counterfactual version of the outcome variable in the post-treatment period.

This procedure is carried out by finding a set of weights for each of the donor pool variables that best predicts the outcome variable. The easiest way to do this is to use linear regression.

In order to decrease the risk of overfitting, the synthetic control estimator forces the weights for the donor pool variables to take values between 0 and 1 and enforces the condition that all the weights sum up to 1.

This constraints the model to learn to predict the outcome variable by interpolating between the donor pool variables, effectively reducing the risk of overfitting.

In a traditional synthetic control estimator, this constraint can be achieved by using the constrained optimization scheme (for more details, check out [Facure, 2020](#) in *Chapter 15*).

As we will follow a Bayesian implementation, we'll use a Dirichlet prior to impose the constraints on the weights, rather than constraining the optimization scheme.

Dirichlet distribution is a multidimensional generalization of the beta distribution. Samples from the Dirichlet distribution are bounded between 0 and 1 and sum up to 1 – a perfect match for our overfitting-reducing idea!

Note that to effectively predict the outcome variable from the donor pool variables under these constraints, we need some of our donor pool variables to take values greater than the outcome and some others to take values lower than the outcome.

Intuitively, note that a weighted average of an array of values can never be lower than the minimum of this array and can never be greater than the maximum of this array, assuming that the weights are bounded between 0 and 1.

Let's take a look at *Figure 11.11* once again. Note that the green line is above, while the red and purple ones are below our outcome variable (blue). This is good.

If all your donor pool variables always take values below or above the values of your outcome, a constraint synthetic control estimator won't work. Theoretically, in such a case, you can transform your variables, but this comes with certain drawbacks (see [Abadie, 2021](#) for details).

Ok, we're ready to do some coding!

We'll use CausalPy, a Bayesian library for quasi-experimental data, to implement the synthetic controls estimator. CausalPy builds on top of PyMC, a popular Python probabilistic programming framework that makes building and debugging Bayesian models seamless.

You can find the code for this section in the `Chapter_11.2.ipynb` notebook (https://bit.ly/causal-ntbk-11_2). Note that this notebook uses a separate conda environment. You'll find installation instructions in the repository's description or the notebook itself (whichever is more convenient for you).

Let's import the libraries:

```
import pandas as pd
import causalpy as cp
import matplotlib.pyplot as plt
```

Let's read in the data:

```
data = pd.read_csv(r'./data/gt_social_media_data.csv')
```

Figure 11.13 presents the first five rows of the dataset:

	date	twitter	linkedin	tiktok	instagram
0	2022-05-15	55	9	23	59
1	2022-05-16	54	18	20	59
2	2022-05-17	54	20	23	57
3	2022-05-18	54	20	21	55
4	2022-05-19	49	23	21	52

Figure 11.13 – The first five rows of the social media search volume dataset

The first column stores the information about the date. The remaining four columns contain information about the relative search volume for a given day.

To make our work with CausalPy smooth, we'll use `date` as the index of our data frame:

```
data.index = pd.to_datetime(data['date'])
data = data.drop('date', axis=1)
```

First, we cast it to the `pandas.Timestamp` type and overwrite the current index.

Second, we drop the original column (the date information is now stored in the index).

Let's plot the data. Figure 11.14 presents the data, with the date of Musk's tweet marked by a black dashed line:

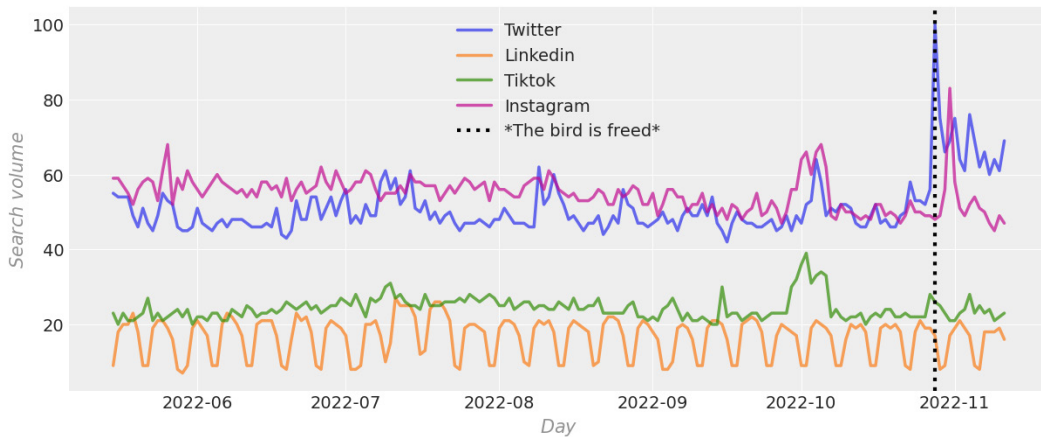


Figure 11.14 – Social media platforms search volume data

Note that Instagram’s search volume is typically higher than that of Twitter, while LinkedIn and TikTok have lower search volumes compared to the latter. Twitter’s line occasionally exceeds Instagram’s line, which is not ideal, but as this happens rarely, we’ll accept it, assuming that it won’t hinder the model’s ability to learn a useful representation.

To prepare the dataset for modeling, we need to store the treatment date in the same format as the dates in the index of our data frame. We will use the `pd.to_datetime()` function for this purpose:

```
treatment_index = pd.to_datetime('2022-10-28')
```

To implement the synthetic controls estimator in CausalPy, we’ll use the `WeightedSumFitter()` class from the `pymc_models` module:

```
model = cp.pymc_models.WeightedSumFitter()
```

To define the model structure, we’ll use the R-style regression formula (you can refer to *Chapter 3* for a refresher on R-style formulas):

```
formula = 'twitter ~ 0 + tiktok + linkedin + instagram'
```

The formula says that we’ll predict `twitter` using the remaining three variables. Zero at the beginning of the formula means that we’ll not fit an intercept.

Finally, we can fit the model. In CausalPy, we do it using the `SyntheticControl` class from the `pymc_experiments` module:

```
results = cp.pymc_experiments.SyntheticControl(
    data,
    treatment_index,
    formula=formula,
```

```
model=model,
)
```

We pass the data, treatment date index, formula, and model to the constructor and assign the object to the `results` variable.

The training will start automatically.

We can now use the fitted `SyntheticControl` object to access useful information about the model.

First, let's plot the results:

```
results.plot(plot_predictors=True)
```

Figure 11.15 presents the output:

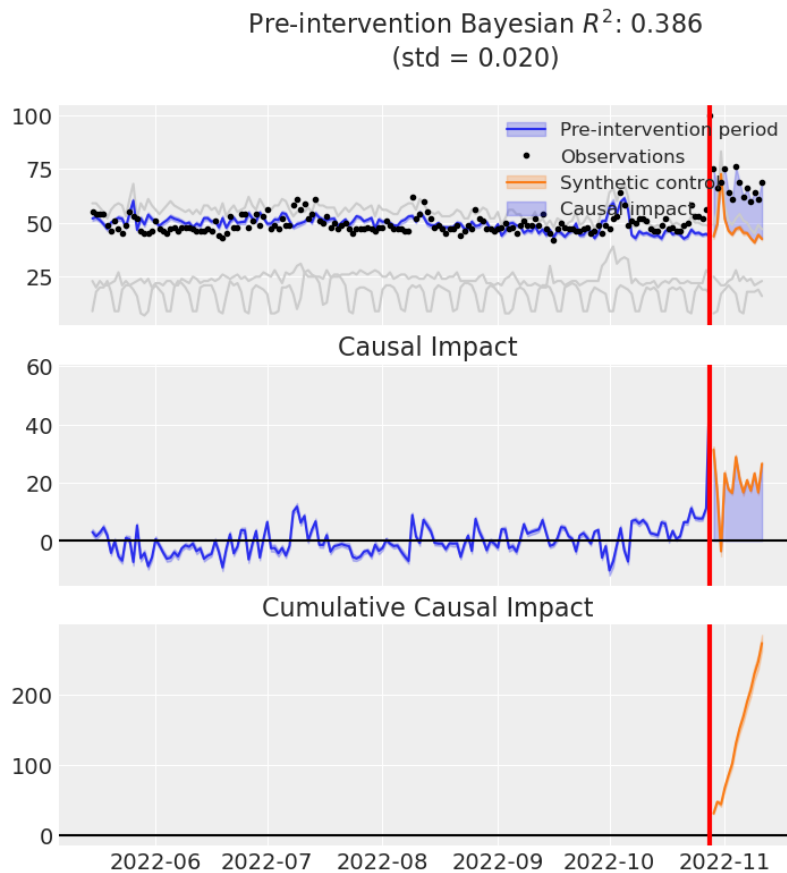


Figure 11.15 – The results of the fitted model

At the top of *Figure 11.15*, we can see the information about the pre-treatment model fit, expressed in terms of Bayesian R^2 (Gelman et al., 2018), which defines how the amount of variability in the outcome variable in the pre-treatment period is explained by the variability in the donor pool predictors.

Our donor pool predictors only explain 38.6% of the variability in the outcome before the treatment. This means that the fit has limited quality.

The top panel presents the actual values of the outcome variable (the black dots) and the predicted values of the outcome variable (the blue line). The orange line represents the predicted synthetic control, and the blue shaded area represents the estimated causal effect of the treatment. The treatment is represented as a red vertical line.

The middle panel presents the difference between the predicted and actual values of the outcome variable. If we had a model that predicted the outcome perfectly, the blue line in the middle panel of *Figure 11.15* would be a straight line fixed at zero in the entire pre-treatment period. This is because a perfect model would have zero errors.

Finally, the bottom panel presents the estimated cumulative causal impact of the treatment.

Let's take a look at how each of the donor pool predictors contributed to the prediction. This can be achieved by calling the `.summary()` method on the `results` object:

```
results.summary()
```

This gives us the following printout:

```
=====Synthetic Control=====
=====
Formula: twitter ~ 0 + tiktok + linkedin + instagram
Model coefficients:
tiktok           0.08, 94% HDI [0.01, 0.18]
linkedin         0.08, 94% HDI [0.01, 0.15]
instagram        0.84, 94% HDI [0.81, 0.87]
sigma            5.79, 94% HDI [5.34, 6.28]
```

The coefficient for Instagram's contribution is the highest (0.84), rendering it the strongest predictor of the outcome.

Note that none of the 94% **highest density intervals (HDIs)** contains zero, suggesting that all predictors were significant. You can think of the HDI as a Bayesian analog of confidence intervals (although this is a simplification; for more details, check out Martin et al., 2021).

The model fit is not perfect (as expressed by R^2), yet the overall effect seems pretty large.

To increase our confidence in the results, we could further formally test the significance of the effect (for more ideas, check out Facure, 2020 in *Chapter 15* and Chernozhukov et al., 2022), but we'll skip this procedure here.

Assuming that our model is correctly specified, we can say that we've found convincing evidence that Elon Musk's tweet increased the Google search volume for Twitter.

That's powerful!

Before we conclude, let's discuss a number of challenges.

Challenges

The donor pool size we used for this analysis is small, which may explain the low value of R^2 . Many practitioners would recommend using at least between 5 to 25 variables in your donor pool as a rule of thumb. That said, smaller donor pool sizes might have certain advantages.

For instance, we can be pretty sure that we're not overfitting, which might happen with larger sizes of donor pools (Abadie, 2021).

We hypothesized that Elon Musk's tweet caused the increase in the search volume for Twitter, yet there might be other factors at work (e.g., media publications on Twitter's acquisition). We might also deal with confounding here.

The Twitter acquisition itself could have caused Musk's tweet *and* increased interest in the information about the platform. This alternative cannot be excluded based on the data alone and might be very difficult to exclude in general.

This consideration shows us an important truth about quasi-experimental methods – they *do not* automatically guarantee causal identifiability.

We should consider all the available data when working with these methods, exactly the same way we do with any other causal method. Considering a DAG that describes our problem, understanding structural relationships between variables, and investigating potential confounding are good first steps in quasi-experimental analysis.

To learn more about good practices regarding data selection and overall synthetic control methodology, check out the excellent papers from Abadie (2021) and Ferman et al. (2020).

Other great resources on synthetic controls include Scott Cunningham's book *Causal Inference: The Mixtape* (Cunningham, 2021) and *Chapter 15* of Matheus Facure's online book *Causal Inference for the Brave and True* (Facure, 2020).

This concludes our section on synthetic control and quasi-experimental methods.

In this section, we introduced quasi-experimental methods and discussed how a synthetic control estimator works. We implemented it in Python using CausalPy and discussed a number of challenges with our analysis.

Wrapping it up

We covered a lot in this chapter. We started by revisiting the S-Learner and T-Learner models and demonstrated how flexible deep learning architectures can help combine the benefits of both models. We implemented TARNet and SNet and learned how to use the PyTorch-based CATENets library.

Next, we delved into the application of causality in NLP. We used a Transformer-based CausalBert model to compute the average treatment effect of a gender avatar on the probability of getting an upvote in a simulated Reddit-like discussion forum.

Finally, we took a glimpse into the world of econometrics and quasi-experimental data and learned how to implement a Bayesian synthetic control estimator using CausalPy.

In the next chapter, we'll start our adventure with causal discovery.

See you on the other side!

References

Abadie, A. (2021). *Using Synthetic Controls: Feasibility, Data Requirements, and Methodological Aspects*. Journal of Economic Literature, 59(2), 391-425.

Abadie, A., & Gardeazabal, J. (2003). *The Economic Costs of Conflict: A Case Study of the Basque Country*. Public Choice & Political Economy Journal.

Benjamens, S., Banning, L. B. D., van den Berg, T. A. J., & Pol, R. A. (2020). *Gender Disparities in Authorships and Citations in Transplantation Research*. Transplantation Direct, 6(11), e614.

Bradbury, J., Frostig, R., Hawkins, P., Johnson, M. J., Leary, C., Maclaurin, D., Necula, G., Paszke, A., VanderPlas, J., Wanderman-Milne, S., & Zhang, Q. (2018). *JAX: composable transformations of Python+NumPy programs [Computer software]*: <http://github.com/google/jax>

Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., ... & Amodei, D. (2020). *Language models are few-shot learners*. Advances in Neural Information Processing Systems, 33, 1877-1901.

Caplar, N., Tacchella, S., & Birrer, S. (2017). *Quantitative evaluation of gender bias in astronomical publications from citation counts*. Nature Astronomy, 1(6), 0141.

Chernozhukov, V., Wuthrich, K., & Zhu, Y. (2022). *A t-test for synthetic controls*. arXiv.

Cunningham, S. (2021). *Causal Inference: The Mixtape*. Yale University Press.

Curth, A., Svensson, D., Weatherall, J., & van der Schaar, M. (2021). *Really Doing Great at Estimating CATE? A Critical Look at ML Benchmarking Practices in Treatment Effect Estimation*. Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks.